Provider APIs Compared: Endpoints, Rate Limits, and Pricing

Introduction

Picking an LLM provider often starts with "which model is smartest" — but the model is only one piece of what you're actually buying. Underneath every provider sits a different endpoint contract, a different rate-limiting policy, and a different pricing meter, and any one of these three can make or break an application long before model quality becomes the bottleneck. This reading works through all three, using the kind of provider landscape you'd actually be choosing from — fast inference specialists, general-purpose labs, routing layers, and newer entrants — so you have a framework for comparing providers on more than benchmark scores.

The three things you're really comparing



A provider decision made on Stage 1 alone — "does it return good text" — skips the two stages that determine whether the integration survives contact with real users.

Endpoint contracts: same job, different shape

Request shape

Some providers expect a flat `messages` array. Others nest system instructions separately from the conversation turns.

Auth pattern

A bearer token in the `Authorization` header is common, but some providers use a custom header key instead. Mixing the two up produces a silent 401 rather than a helpful error.

Response shape

One provider might return a single message string. Another returns a typed array of content blocks (text, tool calls, reasoning) that needs to be unpacked differently.

None of these differences show up in a demo where everything is hardcoded to one provider. They show up the moment a codebase tries to call two providers through the same function.

Rate limits: the ceiling nobody notices until they hit it

Requests per minute (RPM)

Calls allowed per rolling window



Ceiling

Tokens per minute (TPM)

Total token throughput, not call count



Ceiling

Concurrent request limit

Requests in flight at once



Ceiling

A free or low tier on a fast-inference provider might allow generous RPM but a modest TPM — fine for short chat turns, restrictive for a document-summarization workload. A general-purpose provider might do the opposite. The right comparison isn't "which provider has a higher number" — it's "which ceiling does my workload actually bump into first."

Pricing: metered by the token, not the call

Step 1 · Input rate	Step 2 · Output rate	Step 3 · Volume
$1.00 per 1M input tokens	**$4.00** per 1M output tokens (4x input)	**10,000** calls/month · 200 in + 800 out tokens each

Monthly total, 10,000 calls of 200 input + 800 output tokens

Output priced same as input **$10.00**

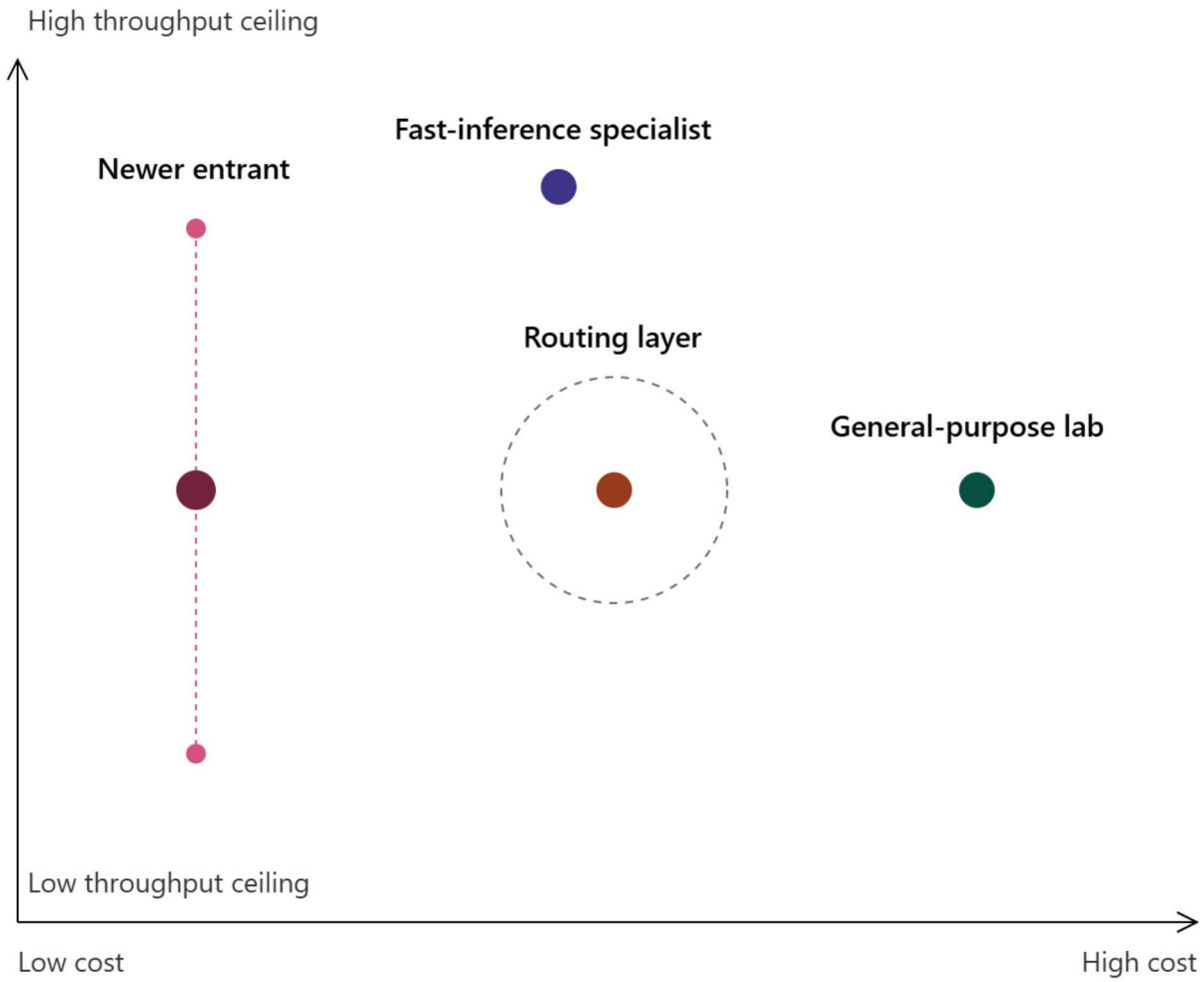


Output at typical 4x input rate **$34.00**



Almost every provider bills per token, but three details change the actual cost: the input-versus-output rate gap, which matters a lot for tasks that generate long responses from short prompts; model-tier pricing, since a single provider often exposes multiple models at different price points and picking the largest by default can multiply cost for a task a smaller model would handle fine; and routing or markup layers, since a provider that routes requests to multiple underlying models may add a pricing layer on top of the underlying model's own rate.

Putting it together: no single "best" provider



A provider optimized for low-latency, high-throughput inference may trade off on per-token pricing at the largest model sizes. A provider offering routing across many underlying models adds flexibility but can obscure which rate limit or price actually applies to a given call. A newer entrant may offer aggressive pricing to win adoption, with rate limits that are generous today and unknown tomorrow. None of these are wrong choices — they're different bets, and the comparison only works if endpoint compatibility, rate ceiling, and pricing are evaluated as three separate questions rather than folded into one gut impression of "which provider felt fastest."

Summary

Endpoint contracts

Auth, request shape, and response shape differ across providers. Those differences surface as integration bugs, not model-quality issues.

Rate limits

Caps apply to request volume, token throughput, or both. The binding constraint depends on the workload, not the provider's marketing.

Pricing

Usage is metered per token, with input and output usually priced differently. Small per-call gaps become large gaps at volume.